CS110 / INDEPENDENCE ROAD

prep / Christopher Guillen

Issued: Nov 10, 2026, 1:37 PM Pacific

Record: PREP-5F0B1FA717F38FA41224A532

Personal public verification code: VIEW-7567690B77A63DE3EFD55B26

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: 7ec03eaf03bda9effd3972a56ca838948e106c695ac5665e8f2fa650baa61af3

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

Personal GitHub repository URL

https://github.com/ChrisGuillen07/week12

Git test: edit run/preview save commit push open remote. Record the test commit and result.

Commit works fine.

Instructor access and team Git readiness: what did you test? Solo: say solo.

Github works fine.

1 Introduction scan notes, one useful idea and one question

Observation: I learned that making mistakes is a normal part of game development. Instead of giving up, developers should learn from their mistakes and try again.

Useful Idea: Every mistake is an opportunity to improve your skills and become a better developer.

Question: What should a developer do when they keep running into problems while making a game?

2 Developer mindset scan notes, one useful idea and one question

Observation: I found the idea of scope creep interesting because it shows how having too many creative ideas can actually hurt a project. Developers might start with a simple game idea but continue adding features until the game becomes too complicated to finish.

Useful Idea: Creating a clear plan before starting development can help prevent unnecessary features from taking over the original idea.

Question: How do developers decide when to stop adding new features and focus on finishing the game?

3 Folders and project creation scan notes, one useful idea and one question

Observation: I found the game development document interesting because it works almost like a blueprint for the entire game. It gives developers an organized place to keep track of the game's concept, requirements, resources, and other important information before they start building it.

Useful Idea: Having a game development document from the beginning can help prevent confusion and keep the project focused on its original idea.

Question: How often should developers update their game development document as new features and ideas are added?

4 Documentation scan notes, one useful idea and one question

Observation: Something I found interesting is that a folder structure can help new people understand a project without having to ask someone where every file is located. Using README files to explain what each folder contains can make it easier for someone to start helping with the game.

Useful Idea: I could include a short README inside folders for things like audio, images, and scripts so another person knows exactly what belongs there and where to find it.

Question: What information should be included in a README so a new team member can understand a project without needing someone to explain everything?

5 Introduction to Unity scan notes, one useful idea and one question

Observation: I learned that deciding where files are stored and how they are backed up should happen early in development. This is important because losing important game assets, such as character models, music, or artwork, could cause a lot of problems.

Useful Idea: Keeping original assets separate from exported versions would help prevent accidentally replacing or losing the files I might need to edit later.

Question: What is the safest way to back up large game files so they can still be recovered if something goes wrong?

6 Import a custom player scan notes, one useful idea and one question

Observation: I found it interesting that game development involves more than just one program. Developers may use different tools for coding, creating 3D models, editing text, communicating, and finding audio resources.

Useful Idea: Keeping links to useful programs and resources in one organized document would save time instead of having to search for the same tools every time I work on the game.

Question: How do developers decide which programs are worth using together instead of trying to do everything in one program?

Your interpretation: what can this pipeline contribute to a prior or new story?

I could use the game development pipeline to create an adventure game based on a futuristic bounty hunter app. In the game, players would take on the role of a bounty hunter who uses an app to find targets, accept missions, and earn rewards. Each mission would have a different target, location, and set of challenges. Some targets might be criminals hiding in crowded cities, while others could be difficult to track down in remote locations. I could use the pipeline to organize mission details, character designs, maps, dialogue, and rewards. As players complete more jobs, they could earn money to upgrade their equipment and unlock harder missions. This would help me create a game with a clear progression system and an interesting world for players to explore.

Questions to resolve before committing to a team or mission

Before committing to a team or mission for my bounty hunter game, I would want to know what everyone's responsibilities are and what the mission requires. I would ask how the team communicates, how rewards are divided, and what happens if someone cannot complete their part. I would also want to know how difficult the mission is, what equipment is needed, and whether there are any risks or unexpected challenges. Having these questions answered beforehand would help prevent confusion, keep everyone working toward the same goal, and make missions run

more smoothly.

Whiteboard

1. Mission System

Players open a bounty app, choose a target, read the target's description, and accept the contract.

